VIDEO LINK→ https://youtu.be/YZak22p0Sns?si=WhZlCZ4rmYk1Z48C

POINT:

Hi everyone. In this project I implemented a 2D Point class that functions as a small vector geometry library with attention to correctness and numerical stability.

The class stores x and y as doubles. I provide constructors, const-correct getters, and chainable setters. The class is designed so operations behave naturally, similar to mathematical vectors.

[operator overloads]

I implemented most arithmetic operators as non-member functions. This allows symmetric operations like Point * scalar and scalar * Point to both work correctly. Addition and subtraction are component-wise, and scalar multiplication scales both coordinates.

For equality, I avoided exact floating-point comparison. Instead, I implemented a tolerance-based helper that combines absolute and relative tolerance, and operator== uses that internally. This prevents fragile comparisons after operations like rotation or normalization.

[dot and cross product]

I implemented both dot and 2D cross product. The dot product supports magnitude checks and projections. The cross product returns the scalar x1*y2 − y1*x2, which is useful for orientation tests and computing signed areas.

[magnitude and normalize]

The magnitude method computes Euclidean length. The normalize method converts a vector to unit length and safely handles the zero vector by leaving it unchanged to avoid division by zero.

[rotate]

Rotation is implemented using the standard 2D rotation matrix. The method accepts degrees, converts to radians, supports clockwise or counterclockwise rotation, and allows rotation about an arbitrary pivot using translation–rotation–translation.

Thank you.

BOX:

This is the Box class. The box is representing a 2D rectangle, and this is designed to store the center position and half dimensions instead of the regular corners. This ensures that the collision detection is in constant time that is, O(1) and this is important because you will have many objects that will be moving around.

OK, so from line 50 and beyond, you'll see that this function does three things, so the position is here so that you could set the position using the center or the bottom-left, and this makes it plain to see what you're controlling something. And then there are transformations. The transformations will help us move the boxes, scale the boxes, and also expand the boxes uniformly. And then the third part is the collision. This collision class will check if two boxes are overlapping, if a box and a circle are colliding, or if a point is inside the box—that is—and it will also calculate distances.

OK, so the reason why this class is important is that it will integrate with a few of our other geometry classes. As you can see, we have a point and a circle class here. And also, importantly, it will provide functionality for the Surface class, like when we want to calculate bounding boxes and then our distances. OK, and then to ensure that this is reliable, there are a few tests that are here—that are all passing as at the point of this recording—so I would say that the Box class is complete and it's ready to support the simulation collision detection needs. Thank you.

SCHEDULER:

[200~Introduction to the Scheduler Class

My name is Joshua, and I will be discussing the Scheduler class. This class allows you to manage and schedule processes—such as agents or features in a simulated world—based on specific priorities.

The Scheduler is a templated class, meaning it is type-agnostic; you can pass in any parameter (e.g., Scheduler<T>) to store and manage various items. It primarily operates in two modes: Deterministic and Probabilistic.

1. Deterministic Mode

In Deterministic mode, the frequency at which an agent runs is strictly tied to its assigned weight.

Setup: If we define three processes with weights of 10, 5, and 1, the scheduler will prioritize them in that exact ratio.

Execution: In a test run of 16 cycles using scheduler.next(), Process 1 (weight 10) ran 10 times, Process 2 (weight 5) ran 5 times, and Process 3 (weight 1) ran exactly once.

2. Probabilistic Mode

The Probabilistic mode uses a random generator to determine the next item based on the weights provided, rather than following a strict sequence.

Example: Using the same weights (10, 5, 1), we ran a simulation of 1,000 cycles.

Results: Process 1 ran 617 times, Process 2 ran 322 times, and Process 3 ran 61 times.

While the distribution aligns with the weights over a large sample size, the specific order is non-deterministic.

3. Starvation Prevention & Auto-Adjustment

A common issue in weighted scheduling is starvation. If Process A has a weight of 100 and Process B has a weight of 1, Process B may almost never get a chance to run.

Without Auto-Adjustment

In a 100-cycle test with a 100:0 weight ratio, Process 1 ran every single time, while Process 2 ran 0 times. Process 2 was effectively "starved" of CPU time.

With Auto-Adjustment

By enabling auto_adjustment, the scheduler ensures a fairer distribution. In the same 20-cycle test, the distribution shifted to 53 calls for Process 1 and 47 calls for Process 2.

How to Configure Fairness:

To implement this, you reset the dynamic weights and enable the following parameters:

Penalty Frequency: The more a process is called, the more it is "penalized" to let others take a turn. A higher value results in a stronger penalty.

Weight Boost Factor: This acts as an "aging" mechanism. If a process stays in the queue too long without being called, its weight is boosted to ensure it eventually runs.

For more details on implementation, you can find additional methods in the scheduler.hpp file. Thank you!

Surface Class - Overview

Overall Design and Goals:

The purpose of the Surface class is to provide a spatial manager for 2D shapes including circle and box objects. It aims to provide overlap detection efficiently and quickly enough with a significant number of objects, while being easy to use and maintain. The class allows for detecting all overlaps between objects and more specific checks.

The key aspects of the program include the ability to work with both Circle and Box Shapes, using sectors to identify overlaps between shapes. The class assigns unique IDs to each shape for fast updating.

The internal structure of the program involves entries for each shape consisting of a shape type, ShapedID, and an occupied_cells vector. Sectors use an unordered map which sends cell keys to shape lists. The program is able to increment next ID values for simple management.

The world is partitioned into cells of specified size. Each shape is inserted into all cells that overlap with its boundary. Collision checks are restricted to shapes that share sectors, which allows for efficient checking. The overall benefits of this design is that it is simple, has predictable patterns of memory access, and provides easy incremental updates.

Many of the key functions implemented by the class include: ShapeID AddCircle(const Circle&) / ShapeID AddBox(const Box&)

bool RemoveShape(ShapeID)

bool UpdateCircle(ShapeID, const Circle&) / bool UpdateBox(ShapeID, const Box&)

bool TranslateShape(ShapeID, const Point&)

std::vector<ShapeID> AllShapeIDs() const

std::vector<ShapeID> QueryRadius(const Point& center, double radius) const
std::vector<pair<ShapeID,ShapeID>> DetectAllOverlaps() const

void SetOverlapCallback(OverlapCallback cb), void step()

void clear()

Example Code Usage:

```
Surface s({.sector_size=5.0});

auto c1 = s.AddCircle(Circle(Point(0,0), 1.0));

auto b1 = s.AddBox(Box(Point(-1,-1), Point(1,1)));

auto nearby = s.QueryRadius(Point(0,0), 2.0);
```

Performance - Depending on the size of the sectors, there is a tradeoff between objects occupying many cells when it is too small, and more shapes per cell when it is too large. Overall, the Surface class is a spatial manager which performs collision detection using sectors and can be effective for reasonably large-sized uses.

CIRCLE:

20:25:09 All right. So this is Lemuel. I work on the cycle class for group 13.

20:25:16 And this is an example usage for the circle and how it could be used. Okay. So this article was made under the namespace ESE 498. So it can be used within the context of this class.

20:25:33 So in this example usage file will have various funds in demonstrating how you can construct the file.

20:25:39 I mean, you can construct a circle from using the centers and then the radius. And also we have an example demonstrating the containment. Okay, so how we contain stuff in the circle.

20:25:53 to notice boundaries, so we don't go, um... over the boundaries we set for the sake of or out of the second nature of the circle. And I also have a function demonstrating, um.

20:26:04 Um, how we can have overlaps and separations. So when two circles come together and they're overlapping, how do we notice the overlapping? And then... If they are not overlapping and they are close to each other, um, how do we know that distance between them and all that?

20:26:21 And also, as you can see here on my 7 to 9.

20:26:24 I have this function demonstrating the distance. Like I said earlier, when there's a distance between you need two cycles that are in close proximity, how do you notice that?

20:26:36 And then there's a function here. That is also demonstrated in circle containment and and we also have an example demonstrating circle interactions when two or more circles are interacting with each other. How do we go? How the code works and all that. So you can see other here.

20:26:54 Um, I like that you are going through the circle and try using it, and then if you find any other functions that you think would be useful for us.

20:27:02 For a group or as a company in general.

20:27:05 Feel free to let me know, and then I can implement this functions. Thank you.